

PROJECT FLOW HANDOVER

Tendso

How the platform works, end to end — for a new engineer, PM, or operator picking this up.

Version 1.0 · 2026-07-06 · Prepared for the Tendso team

1. What Tendso is

Tendso turns a local business into a live, professional one-page website — built from a field agent's phone photos and a short owner interview. A **creator** (field agent) visits a shop, captures photos + an interview, and submits. The platform's AI generates faithful website images and copy, an admin reviews and publishes it, the business owner pays, and the creator gets paid.

The one-line loop: Creator submits → AI generates the site → Admin reviews & publishes → Owner pays → Creator earns.

The four kinds of people in the system

ROLE	WHO	WHERE THEY WORK
Creator (field agent)	Collects business info in the field, earns per approved submission	Mobile app
Admin	Reviews submissions, approves/rejects, publishes sites, manages payouts & creators	Admin web (/admin)
Business owner	The local shop; approves & pays for their site, can request edits	Web (magic link / /my-business)
Prospect / visitor	Learns about Tendso, uses the help chatbot	Public landing site

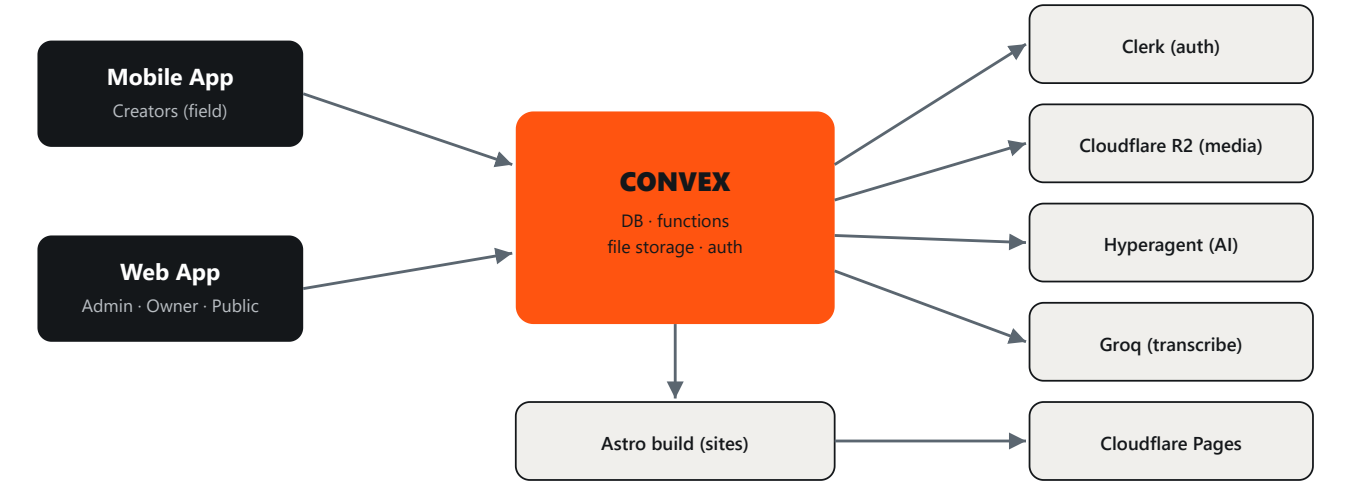
2. Tech stack at a glance

LAYER	TECHNOLOGY	ROLE
Mobile app	Expo / React Native (SDK 54)	Creator's field tool
Web app	Next.js 16 (App Router) on Vercel	Admin, owner portal, landing, help center
Backend	Convex (serverless DB + functions + file storage)	Shared by mobile AND web — single source of truth

Auth	Clerk	Email/password + Google OAuth; passwordless magic link for owners
Media storage	Cloudflare R2	Photos, audio, video (cheap at scale)
Transcription	Groq Whisper	Interview audio/video → text
AI image + copy	Hyperagent (Tendso Studio skill)	Faithful website images + SEO copy — <i>replaced</i> <i>Airtable</i>
Website engine	Astro (template families) → Cloudflare Pages	Builds & hosts the generated sites
Payments	Wise (payouts) · payment link (owner pays)	PHP transfers

Key architectural fact: the **mobile app and the web app share one Convex backend**. A creator submits on mobile; an admin sees it instantly on web — same database, same functions. Neither app has its own server.

3. System map



Both clients talk only to Convex. Convex orchestrates the external services. No separate API server.

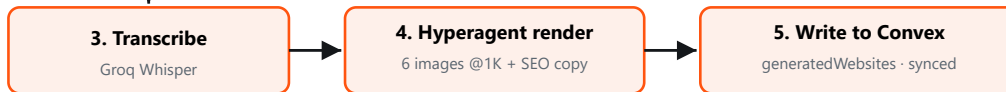
4. The core flow — submission to published website

This is the heart of the product. A creator's submission travels through transcription, AI generation, admin review, publishing, and payment.

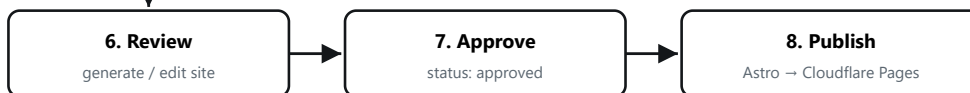
CREATOR (MOBILE)



CONVEX + AI (AUTOMATIC)



ADMIN (WEB)



OWNER + CREATOR



Steps 3–5 are fully automatic. The admin never touches Airtable — the AI writes straight into Convex.

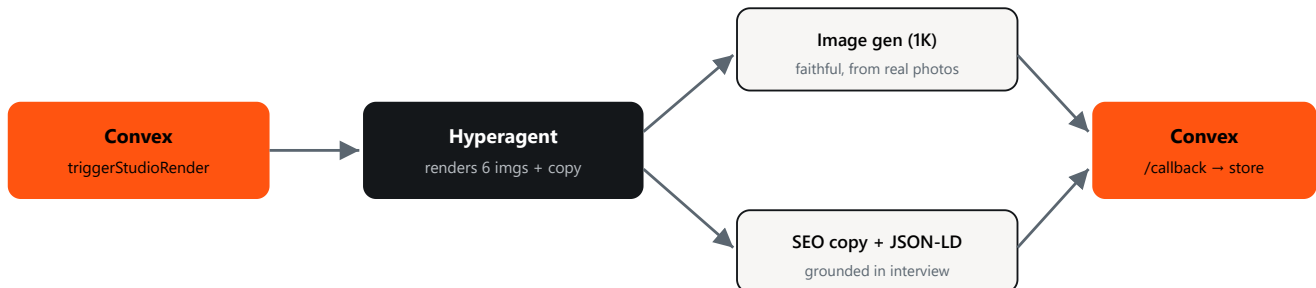
Submission status lifecycle



A submission can also be `rejected` or `unpublished`. The AI-sync sub-status runs in parallel: `pending_push` → `pushed` → `synced`.

5. The AI content pipeline (Hyperagent "Tendso Studio")

This is what replaced Airtable. When a submission is ready, Convex fires a webhook to a **Hyperagent agent** running the *tendso-studio* skill. The agent is a cost-disciplined art director + copywriter.



Images are stored in Convex file storage (permanent URLs). The website builder reads them by section — hero, gallery, about — so the published site uses the AI-optimized photos, never the originals.

Cost discipline (built in): 1K resolution, ~6 images per submission, unusable photos skipped, one job at a time — roughly **\$0.33–0.42 per submission**. Guardrails cap daily volume so cost can't run away.

6. The website engine (templates)

Generated sites are built with **Astro** from reusable **template families**. Each family (e.g. Barbershop, SalonSpa, Autoshop, Restaurant, Shirt Store) has 5 visual variants sharing one section spine (Hero, About, Services, Gallery, FAQ, Location, ...). The admin picks a variant; the builder composes the sections, injects the business's content + AI images, and outputs one self-contained HTML page deployed to Cloudflare Pages.

- A submission's chosen variant is stored as a `heroStyle` code (e.g. `salonspa:K`).
- Editing is live: clicking text/images/links in the admin preview updates the site.
- Images always come from the Hyperagent-optimized set in Convex.

7. Data model — the tables that matter

TABLE	HOLDS
creators	Field agents (+ admins): identity, role, earnings, referral, quiz/approval status
submissions	One business capture: photos, interview, transcript, status, payout

generatedWebsites	The built site: copy, SEO, enhancedImages (AI photos), published URL
businessOwners	Owner portal accounts (passwordless claim)
earnings / payouts	Creator balance, payout requests (Wise)
leads / prospects	Businesses to approach; scraped + tracked
knowledgeArticles / knowledgeQueries	Help-center + chatbot content (RAG); admin trains it on the "Train AI" page
aiKeys	Encrypted BYOK Gemini keys powering the chatbot RAG

8. Auth & roles

Clerk handles identity; Convex reads the Clerk token to authorize every function. A single `creators` record carries a `role` (`creator` or `admin`) — admins are creators with elevated rights. Business owners authenticate via a **passwordless magic link** and live in their own `businessOwners` table.

- **Creator** — signs up on mobile, must pass an onboarding quiz, then an admin approves them before they can submit for pay.
- **Admin** — full access to the `/admin` web console.
- **Owner** — receives a link, reviews their site, pays, requests edits.

9. Deployment topology

PIECE	DEPLOYS VIA	NOTES
Web app (Next.js)	Vercel (from <code>main</code>)	Admin, owner portal, landing, help center
Convex functions	<code>npx convex deploy</code>	Backend for BOTH apps — deploy separately from Vercel
Mobile app	Expo / app stores	Separate repo; shares the prod Convex deployment
Generated sites	Astro build → Cloudflare Pages	Triggered by admin "Publish"

Two deploy targets, one gotcha: backend changes go live via `convex deploy` ; UI changes only go live when merged to `main` and Vercel rebuilds. A change can be "deployed" on the backend but not yet visible in the UI until the Vercel deploy lands.

10. Where to look in the code

YOU WANT TO CHANGE...	START HERE
Submission → AI trigger	<code>convex/submissions.ts</code> (submit), <code>convex/hyperagent.ts</code>
AI callback / image storage	<code>convex/http.ts</code> (<code>/hyperagent-callback</code>), <code>convex/hyperagent.ts</code>
Admin review & publish	<code>app/admin/submissions/[id]/page.tsx</code> , <code>app/api/publish-website</code>
Website build / templates	<code>lib/astro-builder.ts</code> , <code>astro-site-template/src/components/<family>/</code>

Help chatbot / knowledge	<code>convex/knowledgeAI.ts</code> , <code>app/admin/knowledge</code> , <code>components/landing/ChatBot.tsx</code>
Creators / approval	<code>convex/creators.ts</code> , <code>app/admin/creators</code>
Payments / payouts	<code>convex/payments.ts</code> , <code>convex/earnings.ts</code> , <code>app/pay/[token]</code>

Tendso — Project Flow Handover · v1.0 · This document reflects the system after the Airtable→Hyperagent migration. The AI content pipeline now writes directly to Convex; Airtable is no longer in the path.